

Product Requirements Document: Subscription Watchdog

Track: Concierge Agents — Kaggle AI Agents Intensive Vibe Coding Capstone **Build environment:** Google Antigravity **Document version:** 1.0 **Owner:** [Your name / team]
Status: Draft for build

1. Executive Summary

Subscription Watchdog is a multi-agent personal finance assistant that monitors a user's inbox for subscription-related activity — new sign-ups, renewals, and price changes — and takes safe, human-approved action to stop silent money leakage. Unlike passive dashboard apps (WalletFlo, MaxRewards, Kudos) that surface insights for the user to act on manually, Subscription Watchdog closes the loop: it detects, decides against a user-defined policy, drafts the actual cancellation/negotiation email, and only sends after explicit human approval.

The project is built end-to-end inside **Antigravity**, Google's agentic IDE, using the **Agent Development Kit (ADK)** for multi-agent orchestration, **MCP servers** (Gmail, Calendar) for real-world tool access, and a hard-coded human-approval security boundary before any external action is taken.

2. Problem Statement

Recurring subscriptions are structurally designed to be forgotten. Free trials convert silently, prices increase a dollar or two at a time below the threshold of notice, and cancellation flows are deliberately buried. The average user has no systematic way to catch this drift short of manually auditing bank statements every month — a task nobody actually does.

Existing solutions in this space fall into two camps:

1. **Passive trackers** (WalletFlo, MaxRewards, Copilot Money) — surface expiring credits and price changes as notifications, but stop there. The user still has to do the tedious part: find the cancellation page, write the email, remember to follow up.
2. **Card recommendation engines** (Kudos, Pointer, MaxPoints.ai) — solve a different problem entirely (which card to swipe), not subscription lifecycle management.

Gap: nobody closes the loop from "I found a problem" to "I took safe action on it."

Subscription Watchdog is designed specifically to fill that gap, with a security-first design so that closing the loop doesn't mean giving an AI agent unsupervised access to your inbox or your money.

3. Goals and Non-Goals

3.1 Goals

- Detect new subscriptions, renewals, price increases, and likely-unused subscriptions from a user's inbox.
- Apply a transparent, user-editable policy to decide what's worth acting on.
- Draft concrete action (cancellation or price-match negotiation emails) — never send without explicit human approval.
- Maintain a full audit trail of every decision and its reasoning.
- Demonstrate, cleanly and legitimately, at least the following course concepts: multi-agent systems (ADK), MCP servers, security features, deployability, agent skills, and Antigravity-based development.

3.2 Non-Goals (explicitly out of scope for this build)

- No automatic sending of emails or execution of cancellations without human sign-off, ever.
 - No live bank/card account linking (e.g., Plaid) — subscription detection is inbox-based only for this version.
 - No mobile app or polished consumer UI — a CLI or minimal console/Streamlit approval flow is sufficient for the hackathon submission.
 - No multi-user/team support — single-user personal use case only.
-

4. Target User

A single individual who:

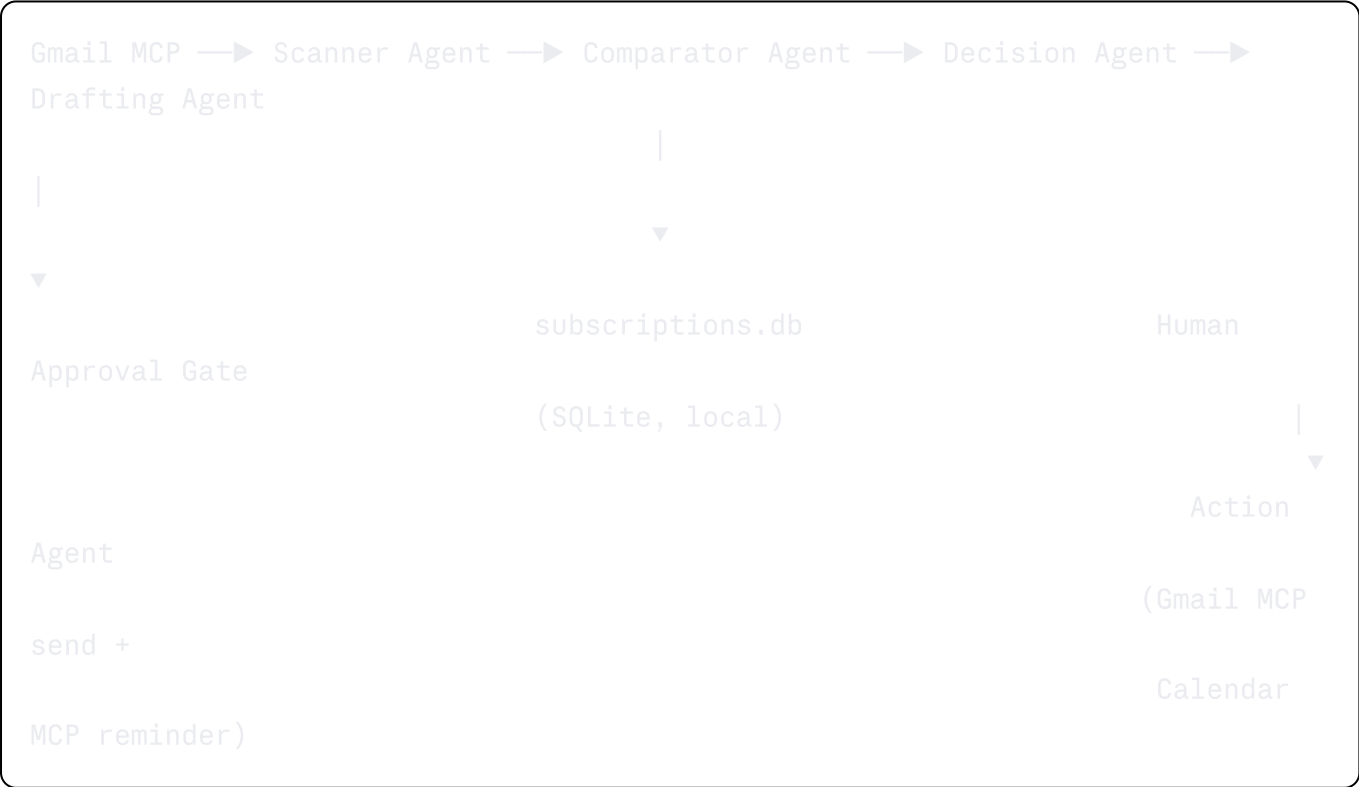
- Has multiple recurring subscriptions (streaming, software, memberships) spread across various merchants.
 - Uses Gmail (or a Gmail-compatible inbox) to receive receipts and renewal notices.
 - Wants to reduce silent subscription creep without spending time auditing it manually.
 - Cares about their financial data privacy — does not want an agent with blanket, unsupervised access to their money or inbox.
-

5. User Stories

#	As a user, I want to...	So that...
1	See a list of all detected subscriptions and their current price	I know what I'm actually paying for
2	Be alerted when a subscription's price increases	I don't get silently overcharged
3	Be alerted when a subscription looks unused	I can decide to cancel it
4	Review a drafted cancellation/negotiation email before it's sent	I stay in control of what leaves my inbox
5	See why the agent flagged something	I can trust and verify its reasoning, not just accept a black box
6	Get a calendar reminder ahead of a renewal date	I have time to act before I'm charged again

6. System Architecture

6.1 High-level flow



An **Orchestrator** (ADK root agent) sequences the pipeline and holds session state across the run.

6.2 Agent responsibilities

Scanner Agent

- Tool: Gmail MCP (`search_messages`), scoped query (e.g. `category:purchases OR subject:(receipt OR renewal OR subscription)`), limited to a configurable lookback window (default: 90 days).
- Extracts structured fields via constrained/structured output: `merchant`, `amount`, `currency`, `billing_cycle`, `next_billing_date`.
- Discards raw email body content immediately after extraction — only structured fields are persisted.

Comparator Agent

- Reads current state from `subscriptions.db`.
- Compares each new scan result against the last known price and billing history for that merchant.
- Produces `Flag` objects for: price increase, new/unrecognized subscription, or subscription unused beyond a configurable threshold (cross-referenced against a user-maintained "still using" list).

Decision Agent

- Applies a policy configuration (see Section 8) to each `Flag` to determine: ignore, log only, or escalate to drafting.
- This is the project's explicit "policy engine" — fully inspectable and user-editable, not hidden in a prompt.

Drafting Agent

- For each escalated flag, generates either a cancellation email or a price-match negotiation email.
- Every draft includes a reasoning trace ("why was this generated") surfaced to the human reviewer.
- Structurally cannot call any send function directly — its only output type is a `Draft` object.

Human Approval Gate

- Minimal CLI (or lightweight console/Streamlit list) presenting each `Draft` with an approve/reject/edit choice.
- No draft proceeds to the Action Agent without an explicit `approved = True` flag set by this gate.

Action Agent

- Executes only on approved drafts.
- Sends the approved email via Gmail MCP `send_message`.
- Creates a Calendar MCP event as a reminder ahead of the next renewal date.

6.3 Antigravity's role in the build

Antigravity is used as the primary development environment for this project:

- Scaffolding the ADK agent structure (root orchestrator + sub-agents) via natural-language-driven "vibe coding" inside Antigravity.
 - Iterating on the Scanner Agent's extraction prompt and structured output schema directly in-editor, with Antigravity surfacing test runs against sample email data.
 - Debugging the MCP tool-calling flow (Gmail/Calendar) using Antigravity's agentic execution trace to inspect what each agent actually called and returned.
 - A short segment of this build process (schema iteration or MCP debugging) will be screen-recorded for the submission video to satisfy the Antigravity evaluation criterion.
-

7. Data Model

```
python
```

```

from dataclasses import dataclass
from datetime import date

@dataclass
class Subscription:
    merchant: str
    amount: float
    currency: str
    billing_cycle: str          # "monthly" | "annual"
    next_billing_date: date
    last_known_amount: float
    still_used: bool | None    # user-declared; None = unknown
    source_email_id: str       # Gmail message id only, never full body

@dataclass
class Flag:
    subscription: Subscription
    reason: str                # "price_increase" | "unused" | "new_subscription"
    severity: str               # "info" | "review" | "action_recommended"
    reasoning_trace: str

@dataclass
class Draft:
    flag: Flag
    draft_type: str            # "cancellation" | "negotiation"
    body: str
    approved: bool = False

```

Storage: local SQLite database (`subscriptions.db`) with three tables: `subscriptions`, `flags`, `audit_log`. No external database dependency, so the project is trivially reproducible from the GitHub repo per the submission requirements.

8. Policy Configuration (Decision Agent)

A single, user-editable config object — deliberately kept out of the LLM prompt so behavior is transparent and auditable:

```
python
```

```

policy = {
    "auto_flag_threshold_pct": 10,      # flag if price rises more than 10%
    "auto_ignore_below_amount": 5.00,  # ignore trivial increases under $5
    "unused_days_threshold": 60,       # flag as "unused" after 60 days
    "draft_action_on": ["price_increase", "unused"],
}

```

9. Security & Privacy Requirements

These are first-class requirements, not an afterthought, and map directly to the "Security features" evaluation concept.

Requirement	Implementation
Human approval before any external action	Hard code boundary: <code>Draft.approved</code> must be <code>True</code> , set only by the Human Approval Gate, before <code>Action Agent</code> can run
Data minimization	Only structured fields (<code>Subscription</code>) are persisted; raw email bodies are discarded post-extraction
Least-privilege MCP scopes	Scanner Agent uses read-only Gmail scope; only Action Agent uses send scope, invoked post-approval
Prompt-injection resistance	Untrusted email content is treated strictly as data during extraction and never concatenated into a prompt context that also has tool-calling ability
Audit trail	Every <code>Flag</code> and <code>Draft</code> , with reasoning trace and timestamp, is written to an <code>audit_log</code> table
No secrets in code	API keys and OAuth tokens loaded via <code>.env</code> , excluded via <code>.gitignore</code> ; never committed to the repo

10. Functional Requirements

1. System shall scan a connected Gmail inbox for subscription-related emails within a configurable lookback window.
2. System shall extract structured subscription data (merchant, amount, cycle, next billing date) from matched emails.
3. System shall persist only structured data, discarding raw email content after extraction.

4. System shall detect price increases, new subscriptions, and likely-unused subscriptions by comparing against stored history.
 5. System shall apply a user-editable policy to determine which flags escalate to drafting.
 6. System shall generate a human-readable draft (cancellation or negotiation email) with an accompanying reasoning trace for each escalated flag.
 7. System shall present all drafts to a human approval gate and shall not send any communication without explicit approval.
 8. System shall send approved emails via Gmail MCP and create a Calendar MCP reminder ahead of the next renewal date.
 9. System shall log every flag, draft, and approval decision to an audit trail.
 10. Decision Agent's policy check shall be packaged as a standalone, CLI-invokable skill (e.g., `agents-cli check-subscriptions`) independent of the full pipeline.
-

11. Non-Functional Requirements

- **Reproducibility:** project must run from a public GitHub repo with clear setup instructions (per submission requirements); no paid/gated dependencies beyond what's "reasonably accessible to all."
 - **Deployability (target, not required to be live):** designed to run as a scheduled job (e.g., daily Cloud Function or cron trigger) rather than requiring manual invocation.
 - **Documentation:** README.md must include problem statement, architecture diagram, setup instructions, and security design rationale.
 - **Code quality:** inline comments explaining agent responsibilities, tool schemas, and security boundaries, per the Technical Implementation grading criterion.
-

12. Evaluation Concept Mapping

Course Concept	How it's demonstrated	Where
Agent / Multi-agent system (ADK)	5-agent pipeline (Scanner, Comparator, Decision, Drafting, Action) with an orchestrator	Code
MCP Server	Gmail MCP (read + send), Calendar MCP (reminders)	Code
Antigravity	Build process (scaffolding, prompt iteration, MCP debugging trace)	Video
Security features	Approval gate, data minimization, least-privilege scopes, injection resistance, audit log	Code + Video

Course Concept	How it's demonstrated	Where
Deployability	Designed for scheduled/cron execution; documented deployment target	Video
Agent skills (Agents CLI)	Decision Agent policy check packaged as standalone CLI skill	Code + Video

13. Milestones (build order, time-boxed)

Step	Task	Est. time
1	SQLite schema + dataclasses (Subscription, Flag, Draft)	30 min
2	Scanner Agent with Gmail MCP (fallback to sample/mock data if live parsing is unreliable)	1-1.5 hr
3	Comparator + Decision Agent (pure logic, policy-driven)	1 hr
4	Drafting Agent (LLM-generated drafts + reasoning trace)	45 min
5	CLI Human Approval Gate + Action Agent	45 min
6	Audit logging + README + security write-up	1 hr
7	Record demo video (problem, architecture, live run, build-in-Antigravity clip)	45 min

14. Risks & Mitigations

Risk	Mitigation
Live Gmail parsing is noisy/unreliable under time pressure	Fall back to a small, realistic set of sample emails for the demo; note this clearly in the README
Scope creep (trying to hit all 6 concepts perfectly)	Prioritize Multi-agent, MCP, and Security as the core three; treat Deployability, Agent Skills, and Antigravity as lighter-weight video/CLI additions
Judges perceive this as "just another subscription tracker"	Pitch explicitly differentiates on closing the loop (draft + approval + action), not just detection

Risk	Mitigation
Accidental inclusion of API keys/secrets in repo	<code>.env</code> + <code>.gitignore</code> checked before final commit; explicitly confirmed in README

15. Submission Checklist (per competition rules)

- ☐ Kaggle Writeup (≤2,500 words), Concierge Agents track selected
- ☐ Cover image + video attached to Media Gallery
- ☐ Video ≤5 minutes, published to YouTube, covering: problem, why agents, architecture, demo, build process
- ☐ Public project link: GitHub repo with setup instructions (live demo optional)
- ☐ README.md with problem, solution, architecture diagram, setup, security rationale
- ☐ No API keys/passwords committed to code